

全程实战：快速上手

Python + AI 生态



雷丰阳

目录

第1章：Python 语法通关

第2章：包管理与虚拟环境

第3章：Jupyter Lab

第4章：智算平台注册与使用

第5章：HuggingFace 实战

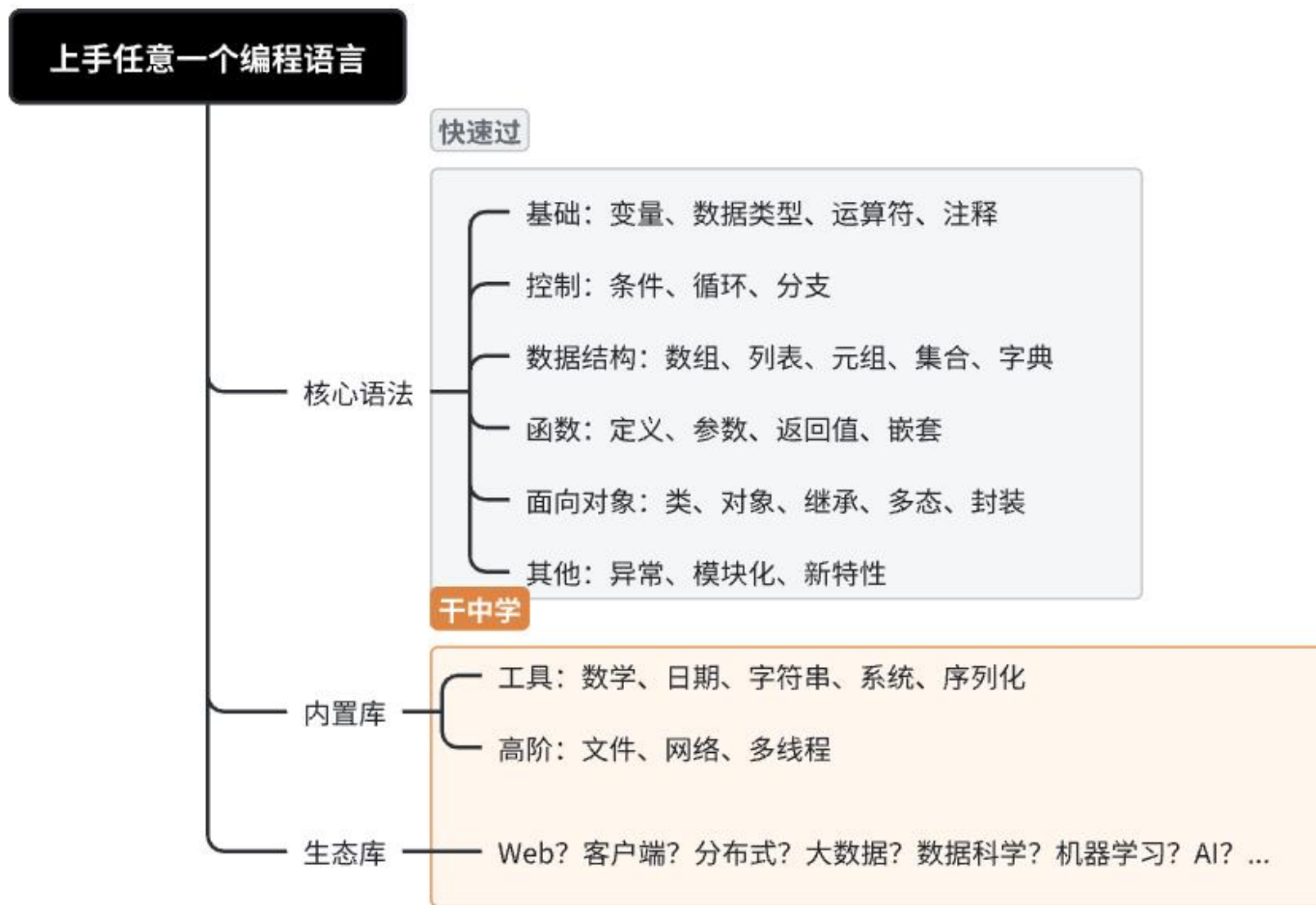
第6章：Gradio 模型可视化

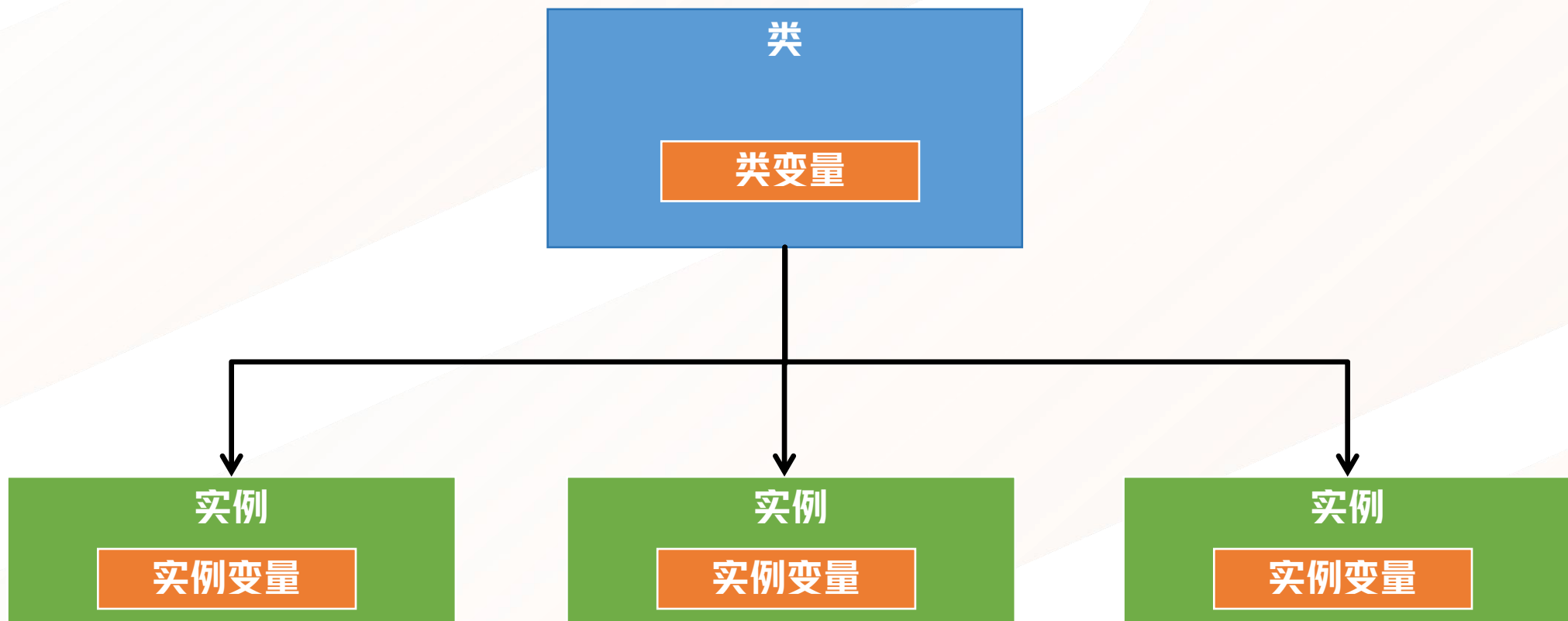
第7章：Flask 编写 REST接口

最后：如何在面试中说自己的技能

第1章

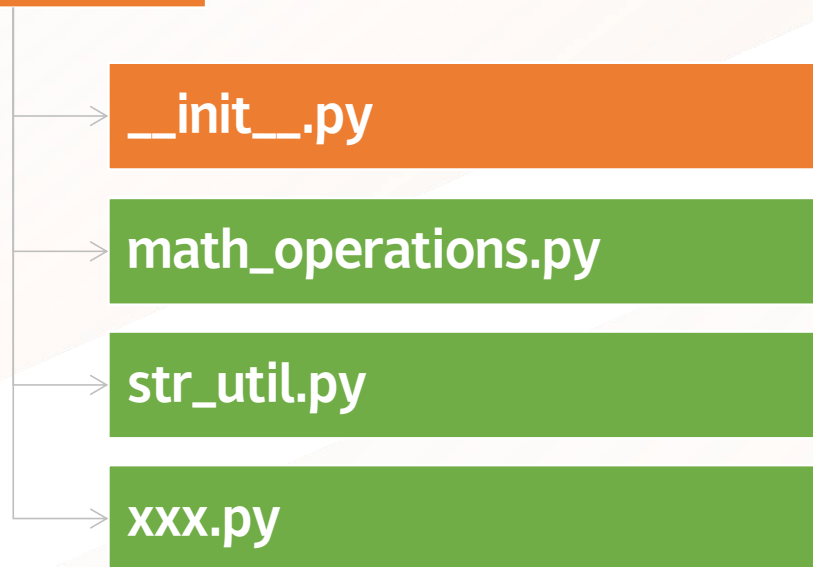
Python 语法通关





2. 模块化

模块：my_pkg



同模块导入：**import** math_operations

跨模块导入：**from** my_pkg **import** math_operations

导入特定函数：**from** my_pkg.math_operations **import** add, sub

3. 速查表



Real Python Pocket Reference

Visit realpython.com to turbocharge your Python learning with in-depth tutorials, real-world examples, and expert guidance.

Getting Started

Follow these guides to kickstart your Python journey:

- realpython.com/what-can-i-do-with-python
- realpython.com/installing-python
- realpython.com/python-first-steps

Start the Interactive Shell

```
$ python
```

Quit the Interactive Shell

```
>>> exit()
```

Run a Script

```
$ python my_script.py
```

Run a Script in Interactive Mode

```
$ python -i my_script.py
```

Learn More on realpython.com/search:

[interpreter](#) · [run a script](#) · [command line](#)

Comments

- Always add a space after the #
- Use comments to explain "why" of your code

Write Comments

```
# This is a comment
# print("This code will not run.")
print("This will run.") # Comments are ignored by Python
```

Learn More on realpython.com/search:

[comment](#) · [documentation](#)

Data Types

- Python is dynamically typed
- Use **None** to represent missing or optional values
- Use **type()** to check object type
- Check for a specific type with **isinstance()**
- **issubclass()** checks if a class is a subclass

Type Investigation

```
type(42)           # <class 'int'>
type(3.14)         # <class 'float'>
type("Hello")     # <class 'str'>
type(True)        # <class 'bool'>
type(None)        # <class 'NoneType'>

isinstance(3.14, float) # True
issubclass(int, object) # True - everything inherits from object
```

Type Conversion

```
int("42")           # 42
float("3.14")       # 3.14
str(42)             # "42"
bool(1)             # True
list("abc")          # ["a", "b", "c"]
```

Learn More on realpython.com/search:

[data types](#) · [type checking](#) · [isinstance](#) · [issubclass](#)

Variables & Assignment

- Variables are created when first assigned
- Use descriptive variable names
- Follow **snake_case** convention

Basic Assignment

```
name = "Leo"        # String
age = 7             # Integer
height = 5.6        # Float
is_cat = True       # Boolean
flaws = None        # None type
```

Parallel & Chained Assignments

```
x, y = 10, 20       # Assign multiple values
a = b = c = 0       # Give same value to multiple variables
```

Augmented Assignments

```
counter += 1
numbers += [4, 5]
permissions |= write
```

Learn More on realpython.com/search:

[variables](#) · [assignment operator](#) · [walrus operator](#)

Strings

- It's recommended to use double-quotes for strings
- Use **"\n"** to create a line break in a string
- To write a backslash in a normal string, write **"\\"**

Creating Strings

```
single = 'Hello'
double = "World"
multi = """Multiple
line string"""
```

String Operations

```
greeting = "me" + "ow!" # "meow!"
repeat = "Meow!" * 3     # "Meow!Meow!Meow!"
length = len("Python")  # 6
```

String Methods

```
"a".upper()           # "A"
"A".lower()           # "a"
"a ".strip()          # "a"
"abc".replace("bc", "ha") # "aha"
"a b".split()         # ["a", "b"]
"-".join(["a", "b"])  # "a-b"
```

String Indexing & Slicing

```
text = "Python"
text[0]   # "P" (first)
text[-1]  # "n" (last)
text[1:4] # "yth" (slice)
text[:3]  # "Pyt" (from start)
text[3:]  # "hon" (to end)
text[::2] # "Pto" (every 2nd)
text[::-1] # "nohtyP" (reverse)
```

String Formatting

```
# f-strings
name = "Aubrey"
age = 2
f'Hello, {name}!'           # "Hello, Aubrey!"
f'{name} is {age} years old' # "Aubrey is 2 years old"
f'Debug: {age}'             # "Debug: age=2"

# Format method
template = "Hello, {name}! You're {age}."
template.format(name="Aubrey", age=2) # "Hello, Aubrey! You're 2."
```

Raw Strings

```
# Normal string with an escaped tab
"This is:\tCool." # "This is: Cool."

# Raw string with escape sequences
r"This is:\tCool." # "This is:\tCool."
```

Learn More on realpython.com/search:

[strings](#) · [string methods](#) · [slice notation](#) · [raw strings](#)

Numbers & Math

Arithmetic Operators

```
10 + 3 # 13
10 - 3 # 7
10 * 3 # 30
10 / 3 # 3.3333333333333335
10 // 3 # 3
10 % 3 # 1
2 ** 3 # 8
```

Useful Functions

```
abs(-5) # 5
round(3.7) # 4
round(3.14159, 2) # 3.14
min(3, 1, 2) # 1
max(3, 1, 2) # 3
sum([1, 2, 3]) # 6
```

Learn More on realpython.com/search:

[math](#) · [operators](#) · [built in functions](#)

Conditionals

- Python uses indentation for code blocks
- Use 4 spaces per indentation level

If-Elif-Else

```
if age < 13:
    category = "child"
elif age < 20:
    category = "teenager"
else:
    category = "adult"
```

Comparison Operators

```
x == y # Equal to
x != y # Not equal to
x < y # Less than
x <= y # Less than or equal
x > y # Greater than
x >= y # Greater than or equal
```

Logical Operators

```
if age >= 18 and has_car:
    print("Roadtrip!")

if is_weekend or is_holiday:
    print("No work today.")

if not is_raining:
    print("You can go outside.")
```

Learn More on realpython.com/search:

[conditional statements](#) · [operators](#) · [truthy falsy](#)

3. 速查表

Loops

- `range(5)` generates 0 through 4
- Use `enumerate()` to get index and value
- `break` exits the loop, `continue` skips to next
- Be careful with `while` to not create an infinite loop

For Loops

```
# Loop through range
for i in range(5):    # 0, 1, 2, 3, 4
    print(i)

# Loop through collection
fruits = ["apple", "banana"]
for fruit in fruits:
    print(fruit)

# With enumerate for index
for i, fruit in enumerate(fruits):
    print(f'{i}: {fruit}')
```

While Loops

```
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f'You entered: {user_input}')
```

Loop Control

```
for i in range(10):
    if i == 3:
        continue # Skip this iteration
    if i == 7:
        break # Exit loop
    print(i)
```

Learn More on realpython.com/search:

for loop · while loop · enumerate · control flow

Functions

- Define functions with `def`
- Always use `()` to call a function
- Add `return` to send values back
- Create anonymous functions with the `lambda` keyword

Defining Functions

```
def greet():
    return "Hello!"

def greet_person(name):
    return f'Hello, {name}!'

def add(x, y=10): # Default parameter
    return x + y
```

Calling Functions

```
greet() # "Hello!"
greet_person("Bartosz") # "Hello, Bartosz"
add(5, 3) # 8
add(7) # 17
```

Return Values

```
def get_min_max(numbers):
    return min(numbers), max(numbers)

minimum, maximum = get_min_max([1, 5, 3])
```

Useful Built-in Functions

```
callable() # Checks if an object can be called as a function
dir() # Lists attributes and methods
globals() # Get a dictionary of the current global symbol table
hash() # Get the hash value
id() # Get the unique identifier
locals() # Get a dictionary of the current local symbol table
repr() # Get a string representation for debugging
```

Lambda Functions

```
square = lambda x: x**2
result = square(5) # 25

# With map and filter
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

Learn More on realpython.com/search:

define functions · return multiple values · lambda

Classes

- Classes are blueprints for objects
- You can create multiple instances of one class
- You commonly use classes to encapsulate data
- Inside a class, you provide methods for interacting with the data
- `__init__()` is the constructor method
- `self` refers to the instance

Defining Classes

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f'{self.name} says Woof!'

# Create instance
my_dog = Dog("Frieda", 3)
print(my_dog.bark()) # Frieda says Woof!
```

Class Attributes & Methods

```
class Cat:
    species = "Felis catus" # Class attribute

    def __init__(self, name):
        self.name = name # Instance attribute

    def meow(self):
        return f'{self.name} says Meow!'

    @classmethod
    def create_kitten(cls, name):
        return cls(f'Baby {name}')
```

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return f'{self.name} barks!'
```

Learn More on realpython.com/search:

object oriented programming · classes

Exceptions

- When Python runs and encounters an error, it creates an exception
- Use specific exception types when possible
- `else` runs if no exception occurred
- `finally` always runs, even after errors

Try-Except

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print(f'Result: {result}')
finally:
    print("Calculation attempted")
```

Common Exceptions

```
ValueError # Invalid value
TypeError # Wrong type
IndexError # List index out of range
KeyError # Dict key not found
FileNotFoundError # File doesn't exist
```

Raising Exceptions

```
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age
```

Learn More on realpython.com/search:

exceptions · errors · debugging

Collections

- A collection is any container data structure that stores multiple items
- If an object is a collection, then you can loop through it
- Strings are collections, too
- Use `len()` to get the size of a collection
- You can check if an item is in a collection with the `in` keyword
- Some collections may look similar, but each data structure solves specific needs

Lists

```
# Creating lists
empty = []
nums = [5]
mixed = [1, "two", 3.0, True]

# List methods
nums.append("x") # Add to end
nums.insert(0, "y") # Insert at index 0
nums.extend(["z", 5]) # Extend with iterable
nums.remove("x") # Remove first "x"
last = nums.pop() # Pop returns last element

# List indexing and checks
fruits = ["banana", "apple", "orange"]
fruits[0] # "banana"
fruits[-1] # "orange"
"apple" in fruits # True
len(fruits) # 3
```

Tuples

```
# Creating tuples
point = (3, 4)
single = (1,) # Note the comma!
empty = ()

# Basic tuple unpacking
point = (3, 4)
x, y = point
x # 3
y # 4

# Extended unpacking
first, *rest = (1, 2, 3, 4)
first # 1
rest # [2, 3, 4]
```

3. 速查表

Sets

```
# Creating Sets
a = {1, 2, 3}
b = set([3, 4, 4, 5])

# Set Operations
a | b      # {1, 2, 3, 4, 5}
a & b      # {3}
a - b      # {1, 2}
a ^ b      # {1, 2, 4, 5}
```

Dictionaries

```
# Creating Dictionaries
empty = {}
pet = {"name": "Leo", "age": 42}

# Dictionary Operations
pet["sound"] = "Purrl" # Add key and value
pet["age"] = 7          # Update value
age = pet.get("age", 0) # Get with default
del pet["sound"]        # Delete key
pet.pop("age")          # Remove and return

# Dictionary Methods
pet = {"name": "Frieda", "sound": "Bark!"}
pet.keys() # dict_keys(['name', 'sound'])
pet.values() # dict_values(['Frieda', 'Bark!'])
pet.items() # dict_items([('name', 'Frieda'), ('sound', 'Bark!')])
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[list](#) · [tuple](#) · [set](#) · [dictionary](#) · [indexing](#) · [unpacking](#)

Comprehensions

- You can think of comprehensions as condensed **for** loops
- Comprehensions are faster than equivalent loops

List Comprehensions

```
# Basic
squares = [x**2 for x in range(10)]

# With condition
evens = [x for x in range(20) if x % 2 == 0]

# Nested
matrix = [[i*j for j in range(3)] for i in range(3)]
```

Other Comprehensions

```
# Dictionary comprehension
word_lengths = {word: len(word) for word in ["hello", "world"]}

# Set comprehension
unique_lengths = {len(word) for word in ["who", "what", "why"]}

# Generator expression
sum_squares = sum(x**2 for x in range(1000))
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[comprehensions](#) · [data structures](#) · [generators](#)

File I/O

File Operations

```
# Read an entire file
with open("file.txt", mode="r", encoding="utf-8") as file:
    content = file.read()

# Read a file line by line
with open("file.txt", mode="r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())

# Write a file
with open("output.txt", mode="w", encoding="utf-8") as file:
    file.write("Hello, World!\n")

# Append to a file
with open("log.txt", mode="a", encoding="utf-8") as file:
    file.write("New log entry\n")
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[files](#) · [context manager](#) · [pathlib](#)

Imports & Modules

- Prefer explicit imports over **import ***
- Use aliases for long module names
- Group imports: standard library, third-party libraries, user-defined modules

Import Styles

```
# Import entire module
import math
result = math.sqrt(16)

# Import specific function
from math import sqrt
result = sqrt(16)

# Import with alias
import numpy as np
array = np.array([1, 2, 3])

# Import all (not recommended)
from math import *
```

Package Imports

```
# Import from package
import package.module
from package import module
from package.subpackage import module

# Import specific items
from package.module import function, Class
from package.module import name as alias
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[import](#) · [modules](#) · [packages](#)

Virtual Environments

- Virtual Environments are often called "venv"
- Use venvs to isolate project packages from the system-wide Python packages

Create Virtual Environment

```
$ python -m venv .venv
```

Activate Virtual Environment (Windows)

```
PS> .venv\Scripts\activate
```

Activate Virtual Environment (Linux & macOS)

```
$ source .venv/bin/activate
```

Deactivate Virtual Environment

```
(.venv) $ deactivate
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[virtual environment](#) · [venv](#)

Packages

- The official third-party package repository is the Python Package Index (PyPI)

Install Packages

```
$ python -m pip install requests
```

Save Requirements & Install from File

```
$ python -m pip freeze > requirements.txt
$ python -m pip install -r requirements.txt
```

Related Tutorials

- Installing Python Packages
- Requirements Files in Python Projects

Miscellaneous

Truthy	Falsy
-42	0
3.14	0.0
"John"	""
[1, 2, 3]	[]
("apple", "banana")	()
{"key": None}	{}
	None

Pythonic Constructs

```
# Swap variables
a, b = b, a

# Flatten a list of lists
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [item for sublist in matrix for item in sublist]

# Remove duplicates
unique_unordered = list(set(my_list))

# Remove duplicates, preserve order
unique = list(dict.fromkeys(my_list))

# Count occurrences
from collections import Counter

counts = Counter(my_list)
```

Learn More on [realpython.com/search:](https://realpython.com/search/)

[counter](#) · [tricks](#)

Do you want to go deeper on any topic in the Python curriculum?

At Real Python you can immerse yourself in any topic. Level up your skills effectively with curated resources like:

- Learning paths
- Video courses
- Written tutorials
- Interactive quizzes
- Podcast interviews
- Reference articles

Continue your learning journey and become a Python expert at realpython.com/start-here 📖🚀

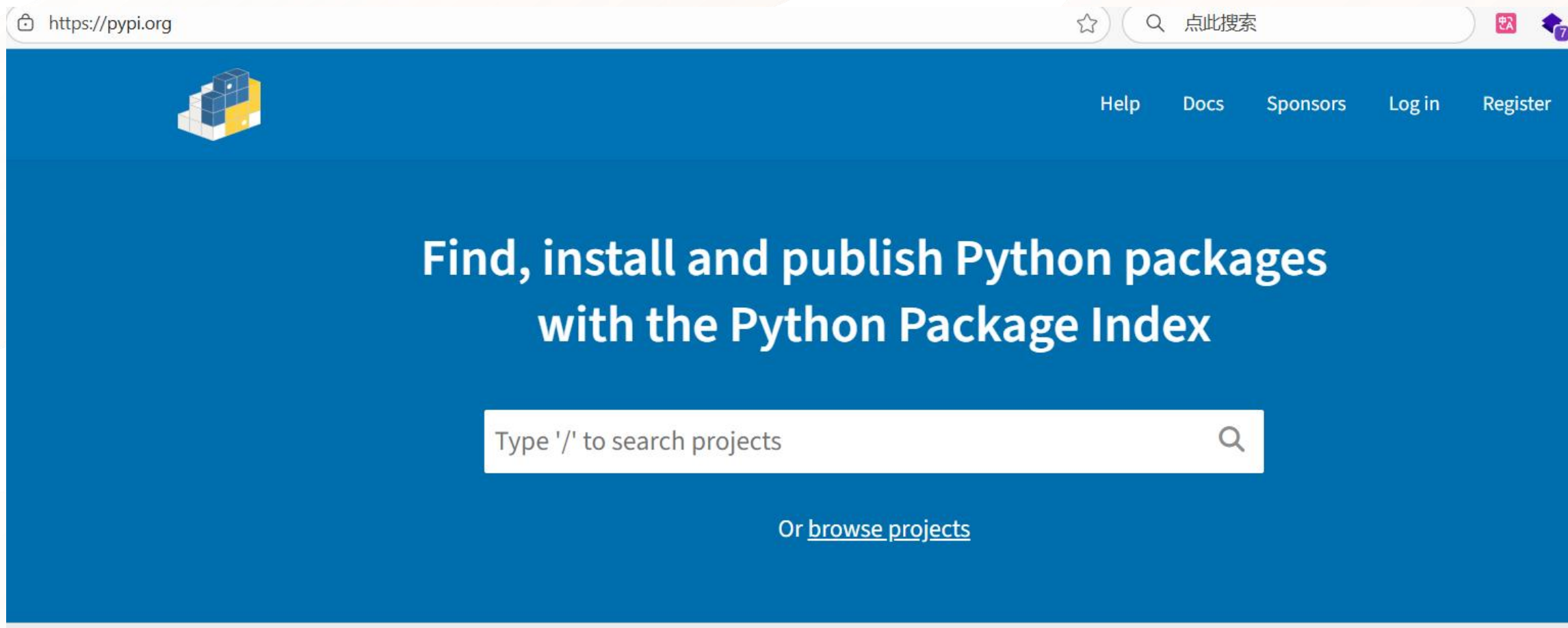
第2章

包管理与虚拟环境

功能描述	命令	说明
安装包	<code>pip install package_name</code>	安装指定的包
安装指定版本	<code>pip install package_name==version</code>	安装包的特定版本
升级包	<code>pip install --upgrade package_name</code>	升级到最新版本
卸载包	<code>pip uninstall package_name</code>	卸载指定的包
列出已安装包	<code>pip list</code>	显示所有已安装的包及其版本
显示包详情	<code>pip show package_name</code>	显示特定包的详细信息
检查可升级包	<code>pip list --outdated</code>	列出所有可以升级的包
导出依赖	<code>pip freeze > requirements.txt</code>	将当前环境的依赖导出到文件
安装依赖	<code>pip install -r requirements.txt</code>	从文件安装所有依赖
清理缓存	<code>pip cache purge</code>	清除pip的缓存
创建虚拟环境	<code>python -m venv venv_name</code>	创建Python虚拟环境（venv模块）

2. Python包仓库

<https://pypi.org/>



配置清华加速源

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

序号	库名	主要功能简介	分类
1	NumPy	科学计算库，支持高效的多维数组和矩阵操作，数学函数丰富	数据分析/科学计算
2	Pandas	数据分析与处理工具，提供 DataFrame、Series 等数据结构	数据分析
3	Matplotlib	绘图与数据可视化库，支持多种图表类型	数据可视化
4	Seaborn	基于 Matplotlib 的高级数据可视化库	数据可视化
5	SciPy	科学计算库，支持数值积分、优化、信号处理等	科学计算
6	Scikit-learn	机器学习库，提供分类、回归、聚类等算法	机器学习
7	TensorFlow	Google 开发的深度学习框架，支持多平台运行	深度学习
8	Keras	高级神经网络 API，封装 TensorFlow 等深度学习框架	深度学习
9	PyTorch	Facebook 开发的深度学习框架，动态图机制易调试	深度学习
10	Requests	HTTP 库，简化网络请求操作	网络请求
11	BeautifulSoup	HTML/XML 解析与网页信息提取	网络爬虫/数据解析
12	Flask	轻量级 Web 框架，适合快速构建网站服务	Web 开发
13	Django	全功能 Web 框架，支持ORM、模板、路由等功能	Web 开发
14	OpenCV	计算机视觉库，支持图像、视频处理与分析	计算机视觉
15	Pillow	图像处理库，支持图像创建、编辑、转换格式	图像处理

创建项目，导入matplotlib，测试如下功能

```
import matplotlib.pyplot as plt
```

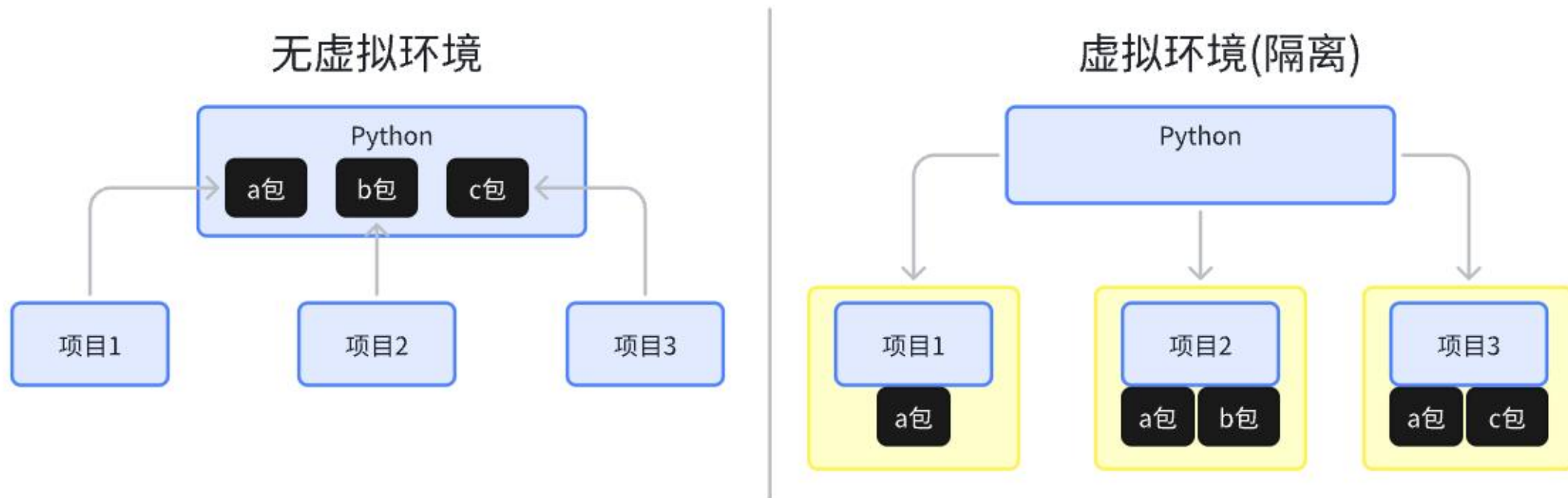
```
x = [1, 2, 3, 4, 5]
```

```
y = [4, 2, 7, 5, 9]
```

```
plt.plot(x, y)
```

```
plt.show()
```

5. 虚拟环境



Anaconda.org

<https://anaconda.org> > anaconda > conda

Conda | [Anaconda.org](https://anaconda.org)

Conda is an open source package management system and environment management system for installing multiple versions of software packages and their dependencies and switching ...

创建虚拟环境

```
python -m venv myenv
```

激活虚拟环境 (Windows)

```
myenv\Scripts\activate
```

停用虚拟环境

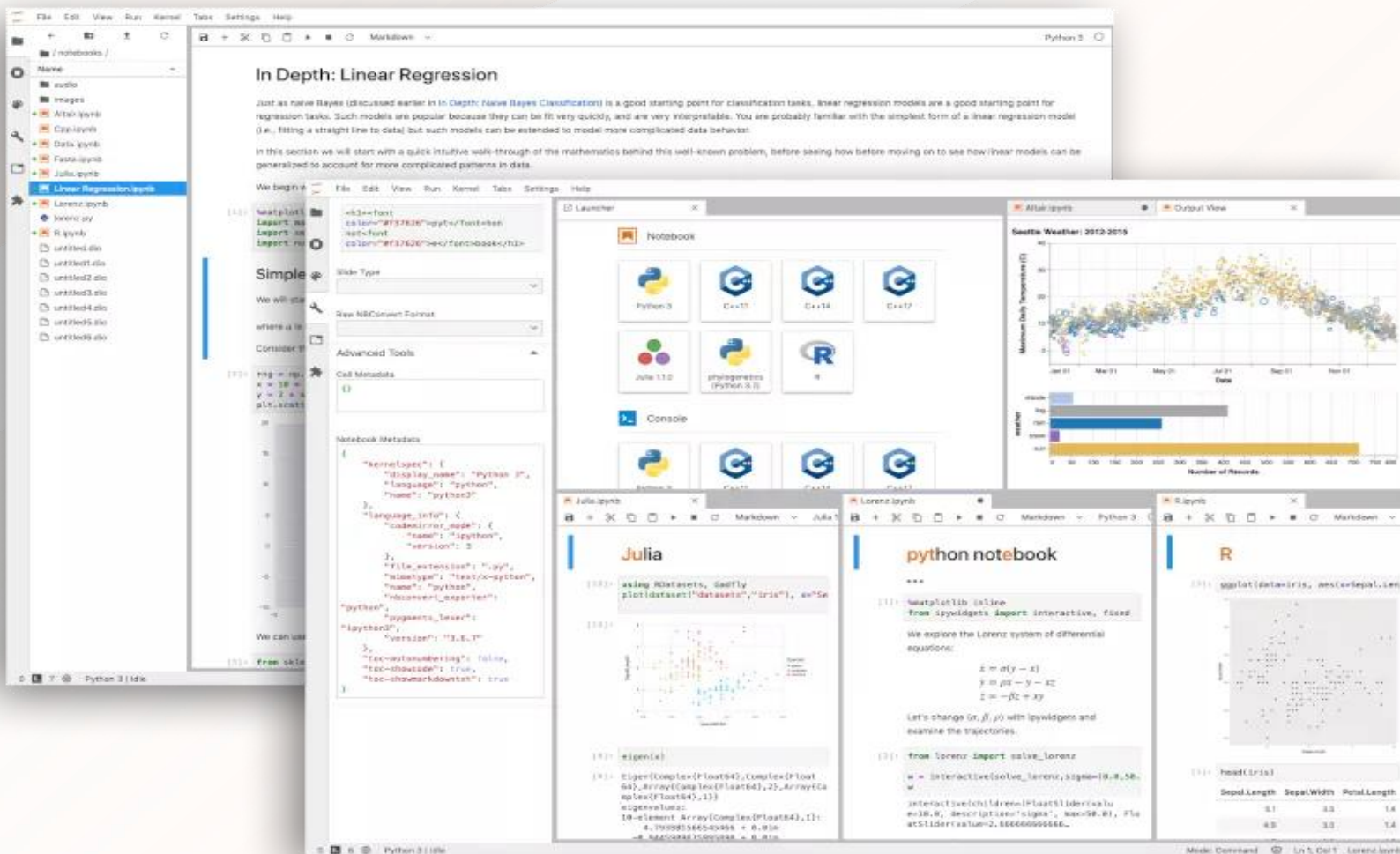
```
deactivate
```

第3章

Jupyter Lab

1. 基本介绍

JupyterLab是最新款基于网络的**笔记本、代码和数据交互式开发环境**。其灵活的界面允许用户配置和安排数据科学、科学计算、计算新闻和机器学习中的工作流程。模块化设计鼓励扩展以扩展和丰富功能。

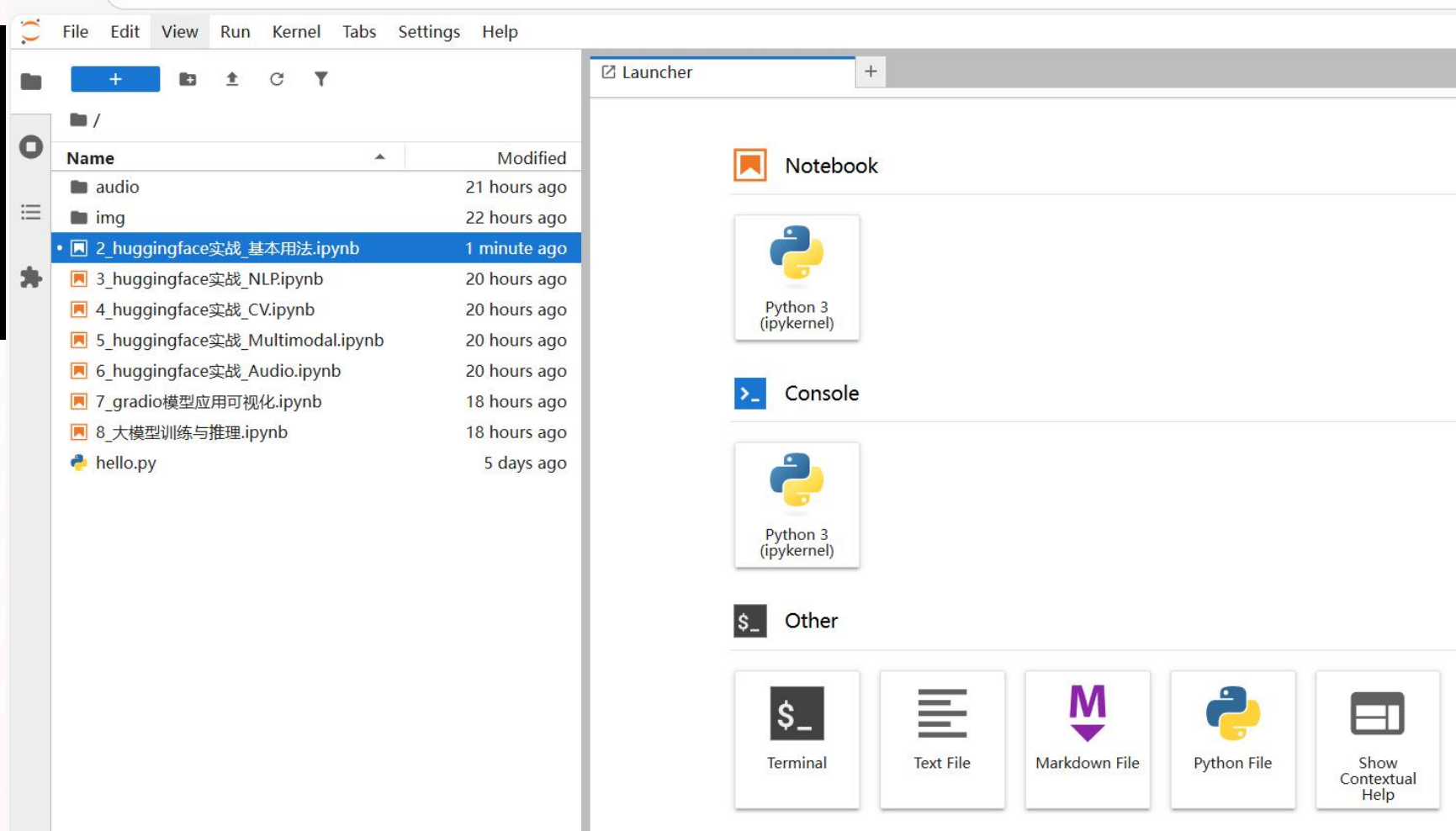


2. 本地安装

<https://jupyter.org/install>

```
# 安装  
pip install jupyterlab
```

```
# 启动  
jupyter lab
```



3. 基本使用



第4章

智算平台注册与使用

1. 大模型三大阶段



为什么用GPU训练深度神经网络?

	Neural Networks	GPUs
天性并行	✓	✓
矩阵计算	✓	✓
浮点计算	✓	✓

- ▶ GPUs 提供 --
 - ▶ 更高的准确性
 - ▶ 耗时更少
 - ▶ 大幅降低计算成本
 - ▶ 降低功耗

GOOGLE DATACENTER



1,000 CPU Servers
2,000 CPUs • 16,000 cores

600 kWatts
\$5,000,000

STANFORD AI LAB



3 GPU-Accelerated Servers
12 GPUs • 18,432 cores

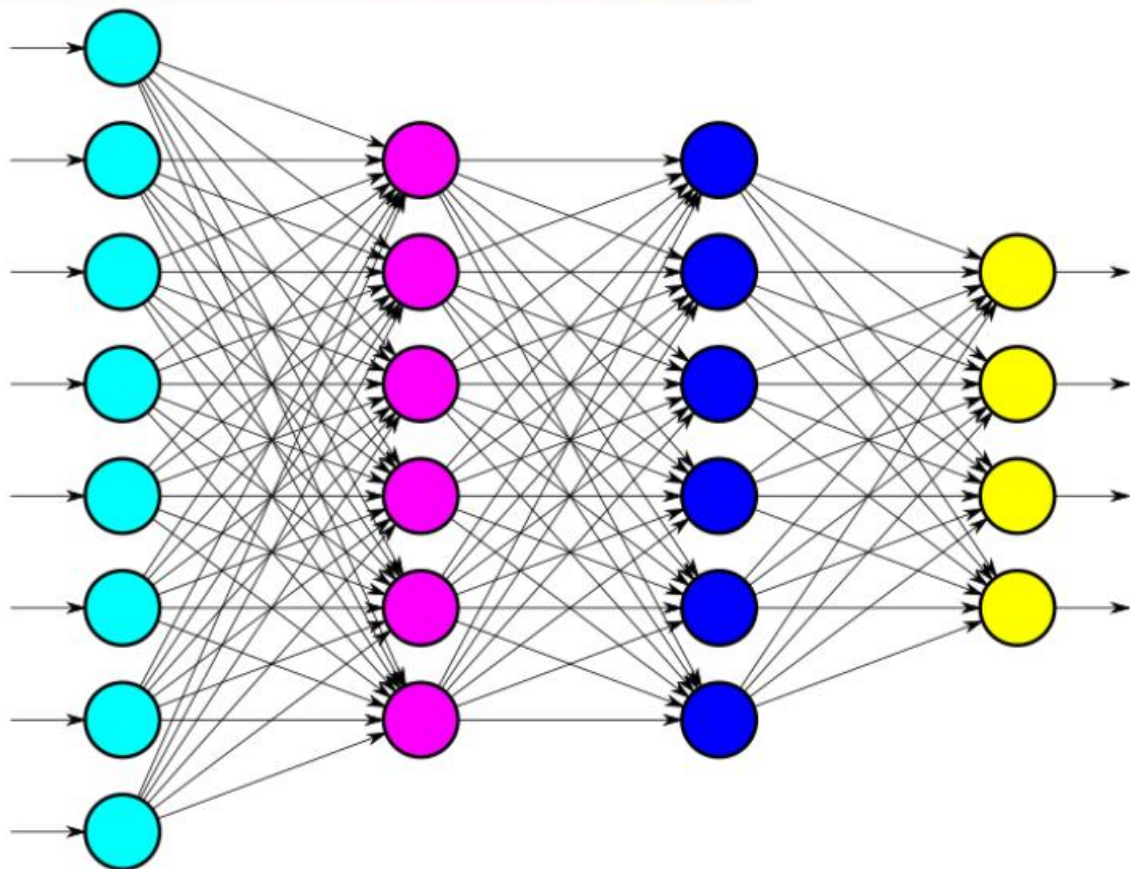
4 kWatts
\$33,000

Now You Can Build Google's
\$1M Artificial Brain on the Cheap

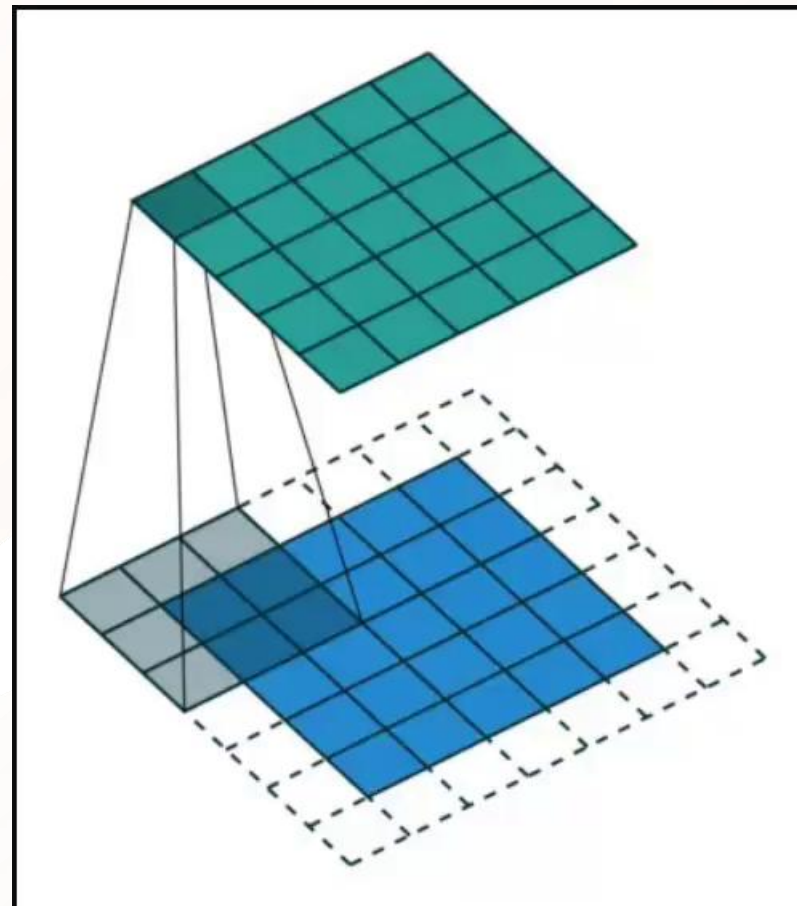
WIRED NVIDIA

3. 机器学习为什么需要显卡

- 研究深度学习和神经网络大都离不开GPU，在GPU的加持下，我们可以更快的获得模型训练的结果
- 计算核心是在给定处理器中实际执行计算的单元，CPU通常有4个、8个或16个核心，而GPU可能有数千个
- 神经网络是高度并行的 (embarrassingly parallel)
- CPU是专家数量少，能力强。GPU是小学生，数量多，只会简单计算



4层深度神经网络



卷积运算示例

4. 注册

访问链接，注册智算平台

https://passport.compshare.cn/register?referral_code=GmZolcTZGzbG0k0S202Zov

第5章

HuggingFace 实战

1. 什么是Hugging Face?

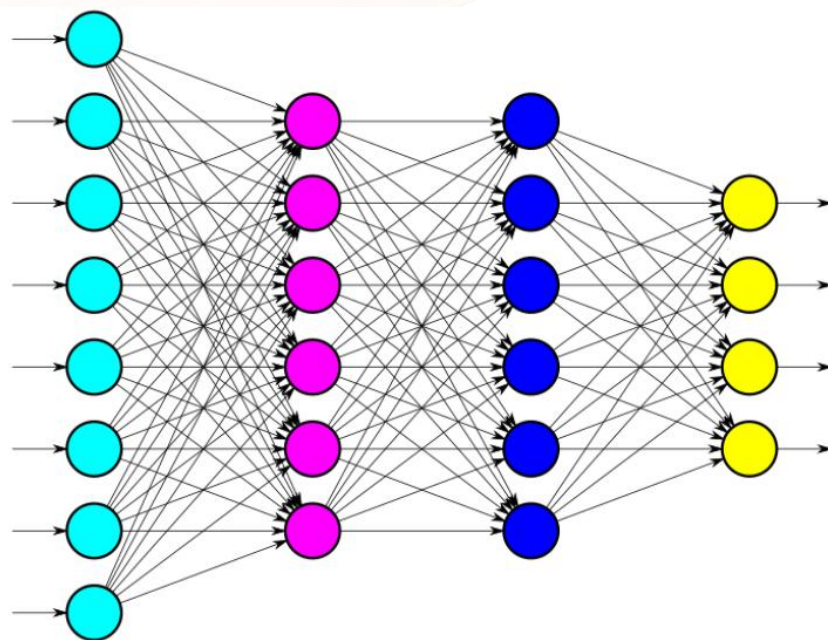
<https://huggingface.co/>

1. 在人工智能开发领域，Hugging Face 已成为一个不可或缺的平台，它让开发者能够轻松调用大量的预训练模型，快速实现各种 NLP 和 CV 任务。
2. Hugging Face: **AI模型的"GitHub"**
3. Hugging Face 是一个用于分享和使用机器学习模型的平台，提供了一个开放的模型共享社区。它不仅提供丰富的预训练模型、强大的工具库，还构建了一个活跃的技术社区



2. 大模型 (Large Model) 定义

1. 大模型 (Large Model) , 通常指的是**参数规模庞大、计算复杂度高的机器学习模型**。
2. 这些模型在训练过程中需要大量的数据和计算资源, 从而能够捕捉到更复杂的模式和关系。
3. 大模型通过增加神经网络的层数和节点数来提高其表达能力。



4层深度神经网络

- GPT4 总共有约 1.8 万亿个参数,分布在 120 个层
- 模型中使用了16 个专家(MoE), 每个专家的 MLP 大约有 1110 亿个参数

MOE: 混合专家 (Mixture of Experts, MoE)

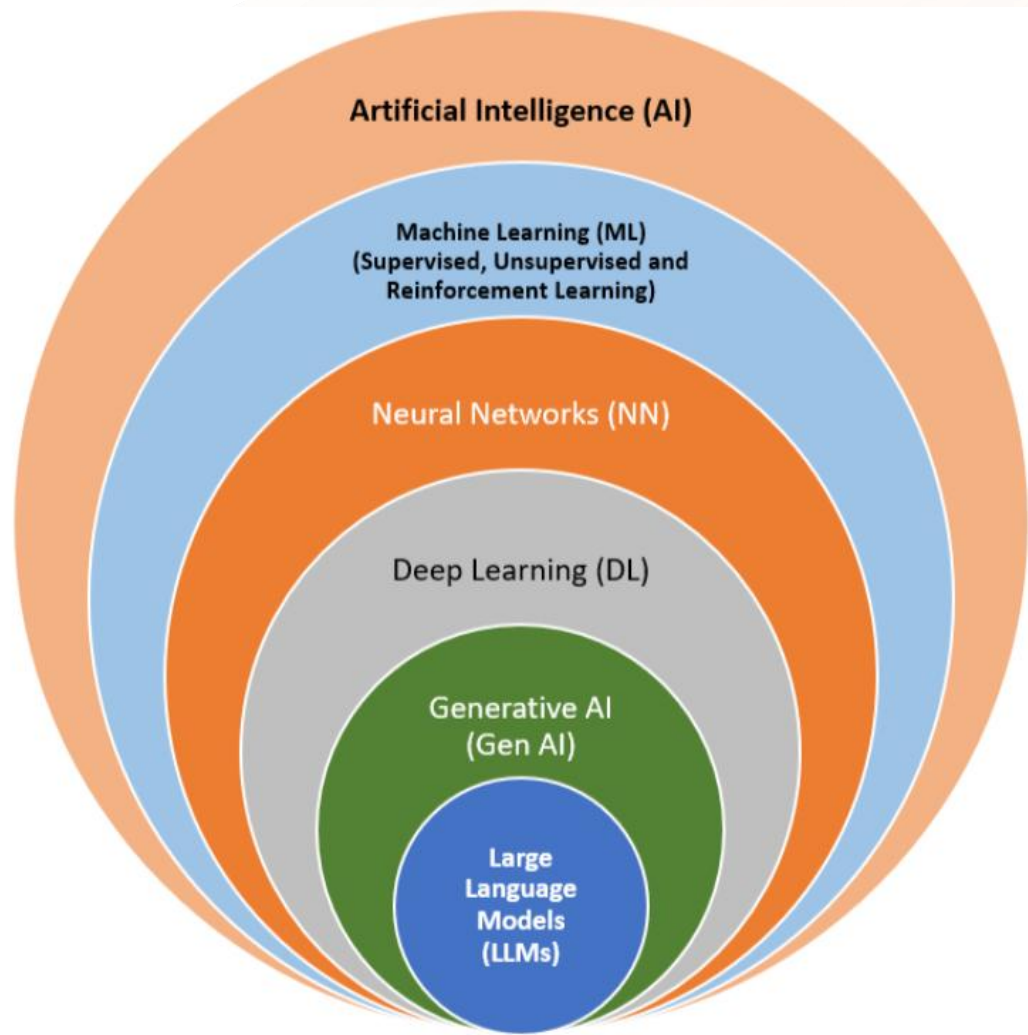
是指同时擅长几个不同领域的专家。就像你可能擅长数学、语文和体育一样, 一个混合专家可能会擅长设计、编程和营销等多个领域。这些专家能够在不同的领域之间进行交流和合作, 从而提供更全面和综合的解决方案。

参考: <https://huggingface.co/blog/zh/moe>

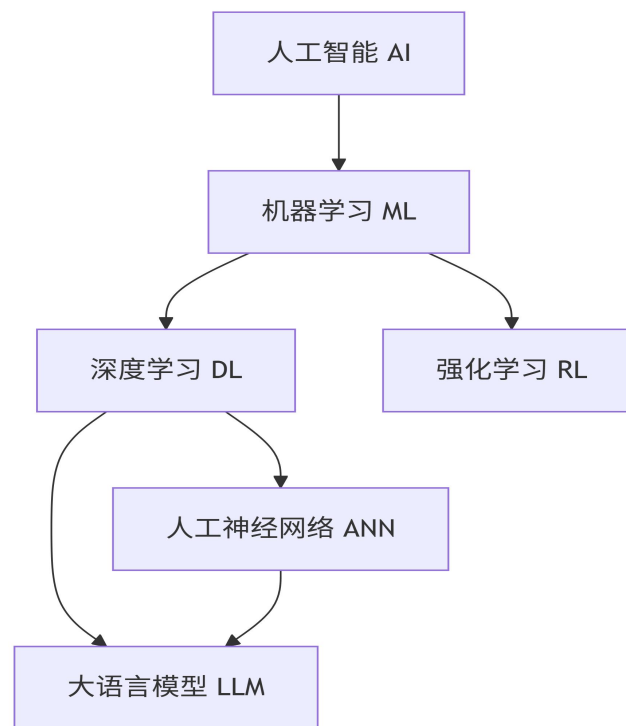
MLP: 多层感知器 (Multilayer Perceptron)

是一种常见的神经网络模型。多层感知器由多个全连接层组成, 每个全连接层接收上一层的所有神经元作为输入, 并将它们作为输出传递给下一层。

3. AI、ML、DL



1. **AI > ML > DL**: 深度学习是机器学习的子集，机器学习是AI的子集。
2. **DL依赖ANN**: 深度学习模型本质是多层神经网络（如CNN、RNN）
3. **LLM是DL的特例**: 大语言模型基于Transformer这一特殊ANN架构。
4. **RL与DL可交叉**: 深度强化学习（如AlphaGo）结合了DL与RL。



4. 大模型分类

Multimodal

- Audio-Text-to-Text
- Image-Text-to-Text
- Visual Question Answering
- Document Question Answering
- Video-Text-to-Text
- Visual Document Retrieval
- Any-to-Any

Natural Language Processing

- Text Classification
- Token Classification
- Table Question Answering
- Question Answering
- Zero-Shot Classification
- Translation
- Summarization
- Feature Extraction
- Text Generation
- Fill-Mask
- Sentence Similarity
- Text Ranking

Computer Vision

- Depth Estimation
- Image Classification
- Object Detection
- Image Segmentation
- Text-to-Image
- Image-to-Text
- Image-to-Image
- Image-to-Video
- Unconditional Image Generation
- Video Classification
- Text-to-Video
- Zero-Shot Image Classification
- Mask Generation
- Zero-Shot Object Detection
- Text-to-3D
- Image-to-3D
- Image Feature Extraction
- Keypoint Detection
- Video-to-Video

Audio

- Text-to-Speech
- Text-to-Audio
- Automatic Speech Recognition
- Audio-to-Audio
- Audio Classification
- Voice Activity Detection

5. 大模型分类 - 按模态（Modality：数据类型）

类型	功能	代表模型	核心机制
Multimodal (多模态)	同时处理、理解和生成多种类型数据（例如文本+图像、语音+文本），实现跨模态任务，如图文匹配、视频问答等。	OpenAI（CLIP）、Salesforce（BLIP系列）、UC Berkeley（LLaVA）	使用 Transformer 架构结合多模态编码器，将不同模态数据映射到共享的向量空间，通过对比学习或联合训练实现跨模态语义对齐。
Computer Vision (计算机视觉)	分析和理解图像或视频内容，实现图像分类、目标检测、语义分割、超分辨率等任务。	Google（ViT）、Facebook（Detectron2）、Ultralytics（YOLOv8）	基于卷积神经网络（CNN）或视觉 Transformer，将像素数据转成特征向量，通过注意力机制或卷积提取空间信息与模式。
Natural Language Processing (自然语言处理)	处理和理解自然语言，包括文本生成、机器翻译、文本分类、问答与信息抽取等。	Google（BERT、T5）、OpenAI（GPT系列）、Meta AI（LLaMA）	多层 Transformer 编码器/解码器结构，利用自注意力机制（Self-Attention）捕捉词之间的上下文关系，通过预训练（如掩码语言模型或自回归语言模型）+微调实现多任务能力。
Audio (音频处理)	处理声音数据，包括语音识别（ASR）、语音合成（TTS）、说话人识别、音频分类等。	Facebook（Wav2Vec 2.0）、OpenAI（Whisper）、Google（Tacotron 2）	利用卷积层、Transformer 或端到端声学模型，将原始波形或频谱特征编码成高维表示，通过连接解码器或预测网络实现序列到序列转换。

6. 大模型分类 - 按任务类型

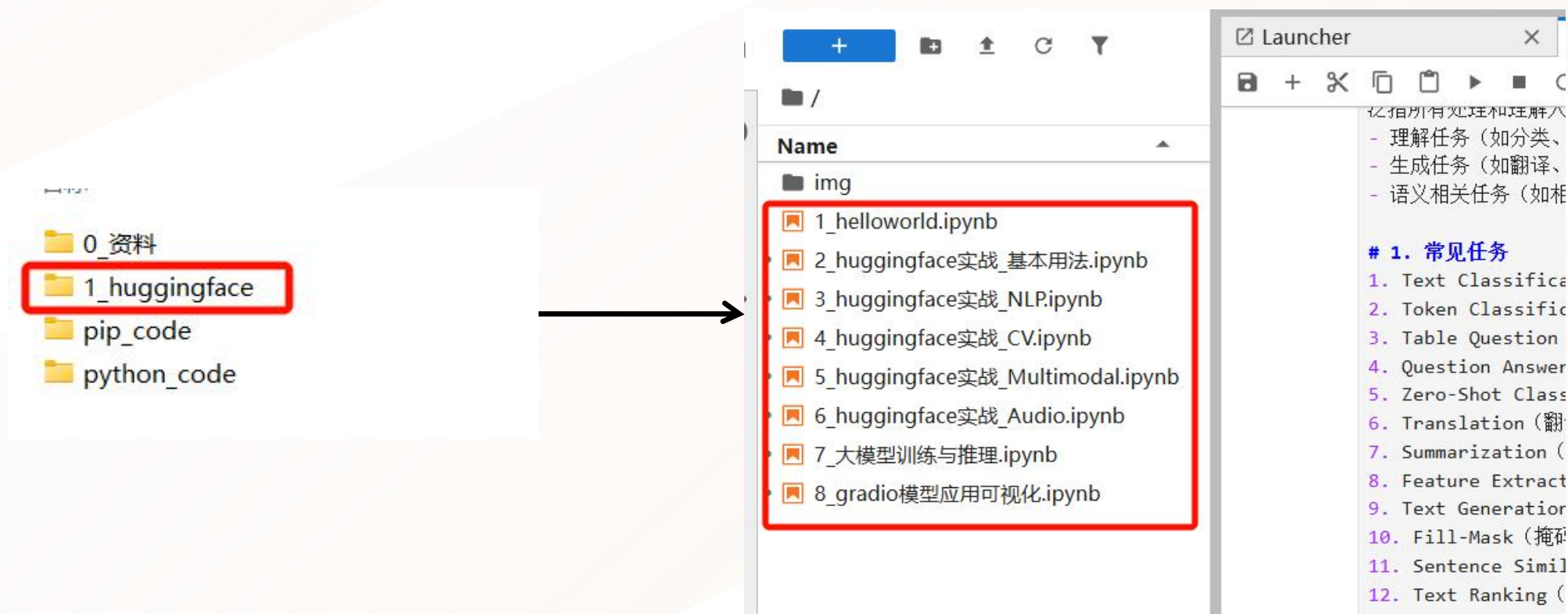
模型类别	功能介绍	代表模型	核心机制
生成式模型 (Generative Models)	根据输入生成新的数据（文本、图像、音频等），用于创作、内容补全、数据增强等。	GPT-4、Stable Diffusion、MusicGen	自回归或扩散模型，通过概率分布采样与条件约束生成高质量样本。
判别式模型 (Discriminative Models)	对输入数据进行分类或判断，如情感分类、图像类别识别等。	BERT、ResNet、DETR	学习类别条件概率或直接学习判决边界，基于特征提取+分类器。
混合结构 (Encoder-Decoder)	同时编码输入特征并解码输出，常用于翻译、摘要、图像描述等。	T5、MarianMT、BLIP	Transformer 编码器-解码器架构，支持多任务学习与跨模态生成。

7. 大模型分类 - 按学习方式

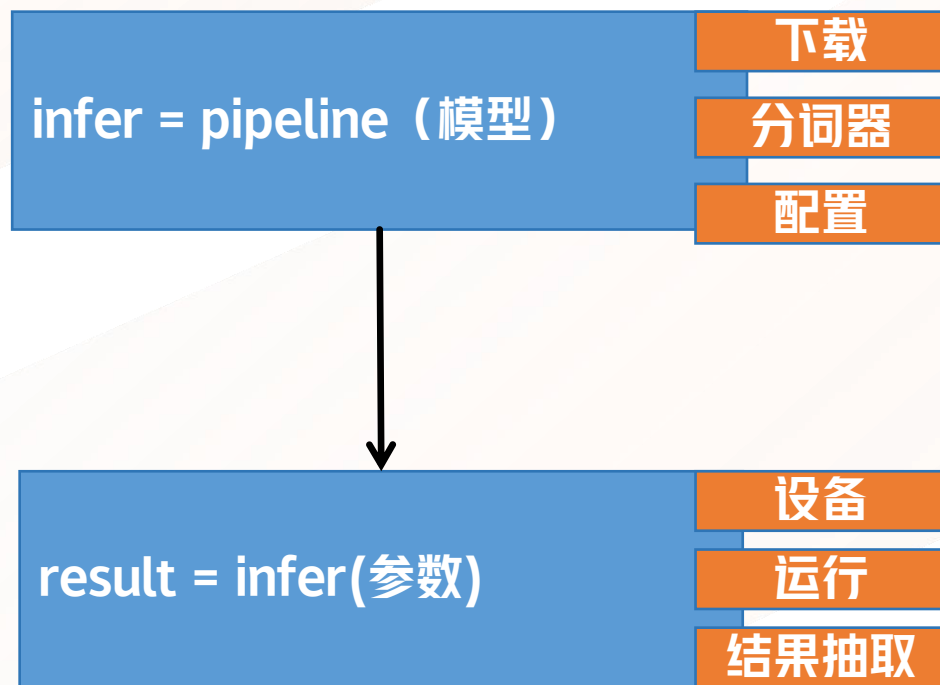
模型类别	功能介绍	代表模型	核心机制
监督学习大模型	在大量标注数据上训练，精确拟合输入输出模式。	ViT、RoBERTa	基于标注数据最小化损失函数，优势是可控，劣势是数据成本高。
自监督学习大模型	利用数据内部结构（如掩码预测、对比学习）进行预训练。	BERT、SimCLR、wav2vec 2.0	自动生成训练目标，从海量无标注数据中学习特征表示。
强化学习大模型	在动态环境中通过奖励信号优化策略，适用于决策、交互任务。	AlphaGo、RLHF-GPT	基于策略梯度或值迭代，在多轮交互和反馈中优化行为。

模型类别	功能介绍	代表模型	核心机制
Transformer 系列	使用自注意力机制，擅长序列到序列任务。	BERT、GPT、ViT	Attention 捕捉长程依赖，便于并行训练。
Diffusion 系列	基于扩散概率建模，逐步去噪生成高质量样本。	Stable Diffusion、Imagen	逐步添加+去除噪声，学习数据分布。
RNN/LSTM 系列	通过循环连接处理时间序列或较短依赖文本。	LSTM Seq2Seq、Tacotron 2	按时间步迭代更新状态，捕捉时间依赖。

所有实战内容，参考 jupyter lab 笔记



10. pipeline API 基本用法



文本分类 (Text Classification)

文本摘要 (Text Summarization)

命名实体识别 (ner)

图像分类 (Image Classification)

目标检测 (Object Detection)

视觉问答 (Visual Question Answering)

文档问答 (Document Question Answering)

自动语音识别 (ASR)

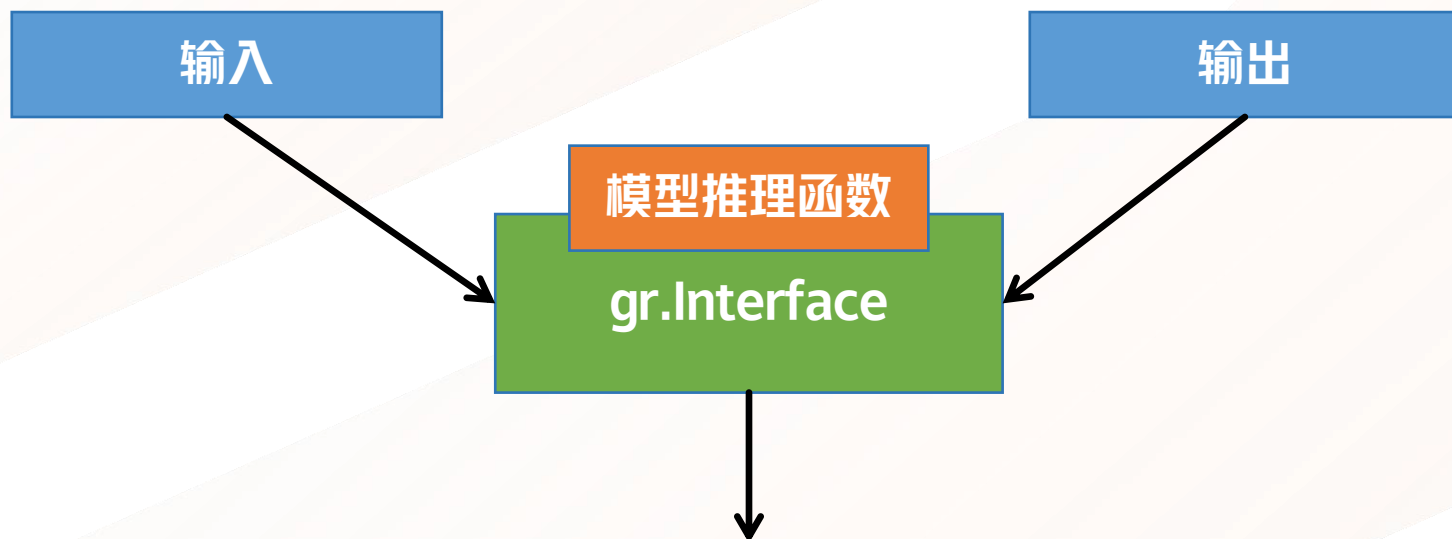
文本转语音 (TTS)

文本转音频 (TTA)

第6章

Gradio 模型可视化

1. Gradio 基本用法



情感分析示例

输入一句话，判断它的情感倾向（简单示例）

请输入文本

我喜欢篮球

情感分析结果

积极 😊

Clear

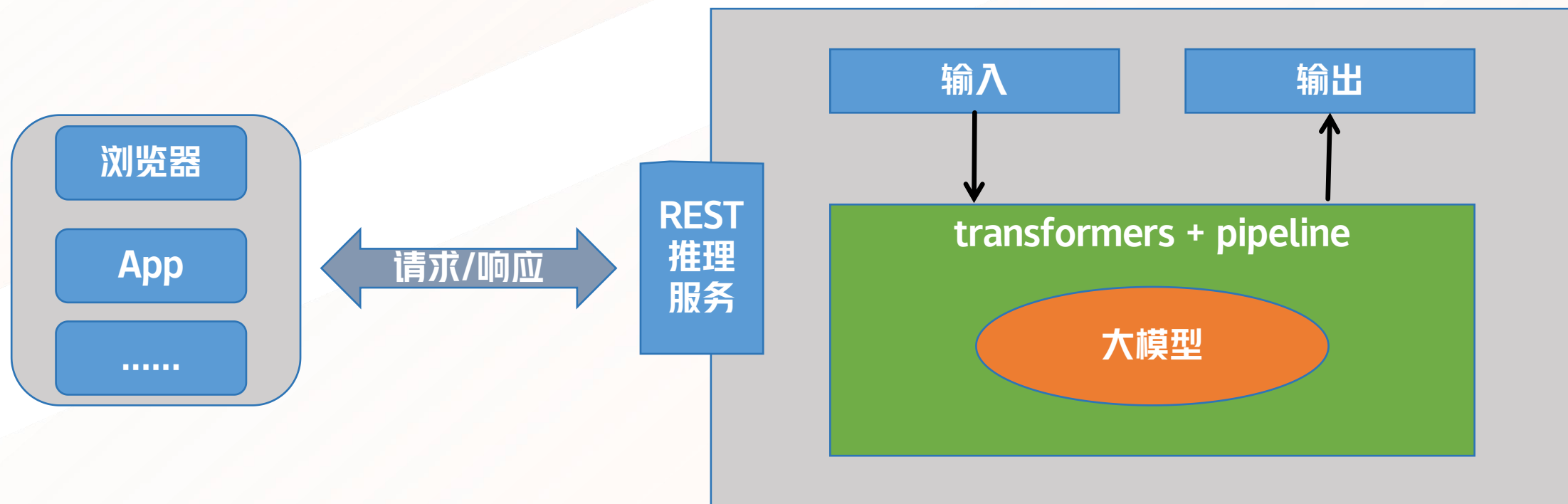
Submit

Flag

第7章

Flask 编写 REST 接口

1. 为什么写 REST 接口



2. Flask 介绍

Flask 是一个用 Python 编写的轻量级 **Web 应用框架**。它被归类为微框架（microframework），因为它不需要特定的工具或库，保持核心简单但可扩展。

```
from flask import Flask, jsonify
```

```
app = Flask(__name__)
```

```
@app.route('/') →
```

请求映射

```
def hello_world():
```

```
    person = {  
        "name": "Claude",  
        "age": 25  
    }
```

```
    return jsonify(person) →
```

响应返回

```
if __name__ == '__main__':
```

```
    print("服务器启动...")
```

```
    app.run(debug=True, host='0.0.0.0', port=5000) →
```

服务启动

3. 获取请求信息

request

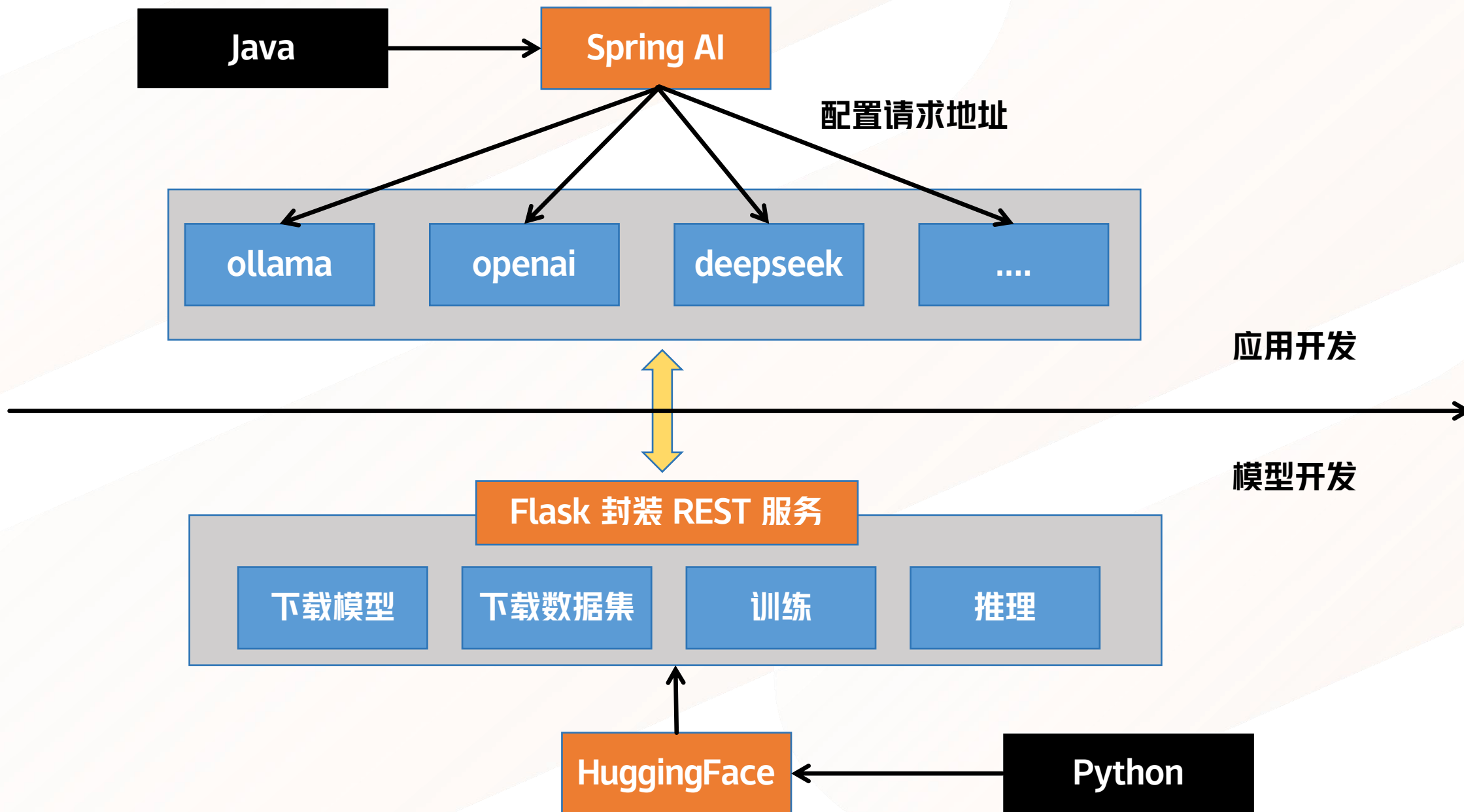
.

请求路径, 请求头, 请求参数, 请求体

```
"method": request.method,  
"url": request.url,  
"base_url": request.base_url,  
"path": request.path,  
"query_string": request.query_string.decode(),  
"remote_addr": request.remote_addr,  
"user_agent": request.user_agent.string,  
"content_type": request.content_type,  
"content_length": request.content_length,  
"is_json": request.is_json,  
"is_secure": request.is_secure,  
"scheme": request.scheme,  
"host": request.host,
```

最后：

如何在面试中说自己的技能



详细版：

1. 熟悉 Python 语法，理解面向对象、模块化、包管理、虚拟环境
2. 熟悉 HuggingFace/ModelScope 社区，熟练使用 多模态、NLP、CV、Audio等各种模型
3. 熟练 Transformers pipeline API 进行模型推理，熟悉模型微调流程
4. 熟练使用 Gradio 构建各种大模型应用
5. 熟悉 Flask Web框架，掌握REST API操作

精简版：

1. 熟悉 Python ， AI生态，大模型微调、推理流程，
2. 熟练掌握 多模态、NLP、CV、Audio等各种的模型功能与方案选型
3. 熟悉 Gradio、Flask、Transformers、PyTorch 等各种框架，快速构建大模型应用

快猫项目

1. 负责大模型基座构建，使用 Python + Transformer API 构建 情感分类大模型 推理服务
2. 使用 Python + Flask 统一封装 AI审核 REST服务，提供多情感标签返回
3. 负责智算平台模型部署 与 推理服务部署运维工作

谢谢观看！